



廈門大學

《计算机组成原理》 课程实验报告

姓名： 芦思瑶

学院： 信息学院

系： 软件工程系

专业： 软件工程

学号： 11920232202561

2025 年 5 月 21 日

第 7 次实验 流水线 CPU 设计

1. 实验目的

- (1) 了解理想流水线 MIPS 处理器设计的基本原理，在该流水线上运行测试程序。
- (2) 了解气泡流水线 MIPS 处理器设计的基本原理，在该流水线上运行测试程序。
- (3) 了解重定向流水线 MIPS 处理器设计的基本原理，在该流水线上运行测试程序。
- (4) 了解动态分支预测流水线 MIPS 处理器设计的基本原理，在该流水线上运行测试程序。

2. 实验环境

- (1) Windows 系统下运行 Logisim 软件（需安装 JDK）。
- (2) MARS 4.5 汇编工具。

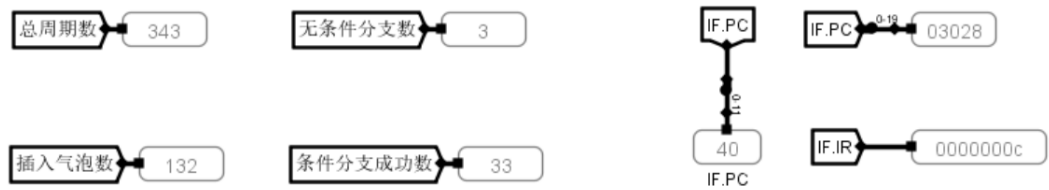
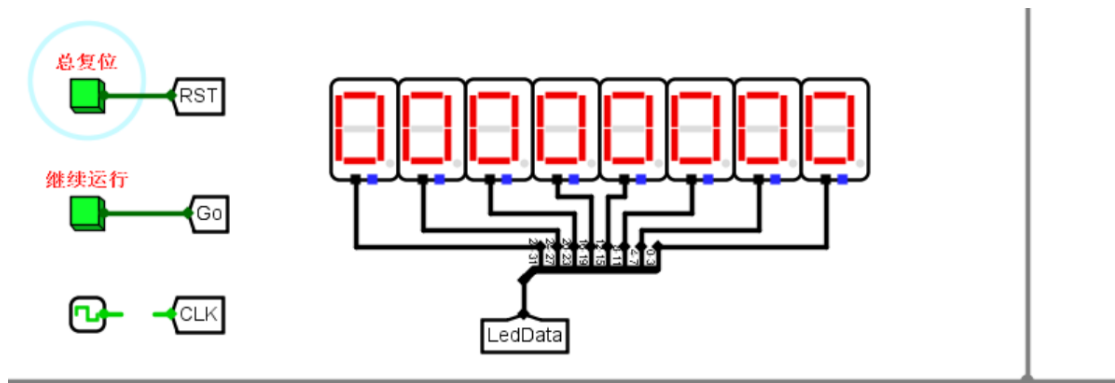
3. 实验内容

- (1) 在理想流水线 MIPS 处理器上运行测试程序。

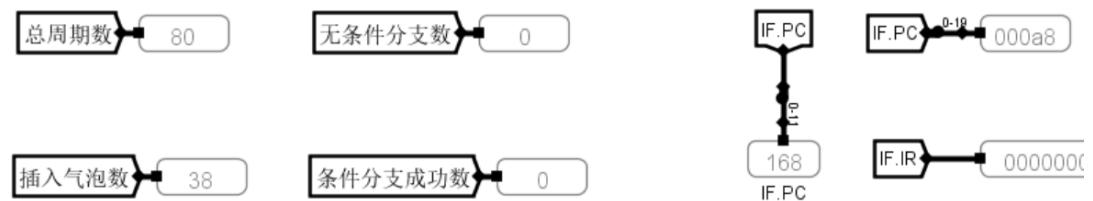
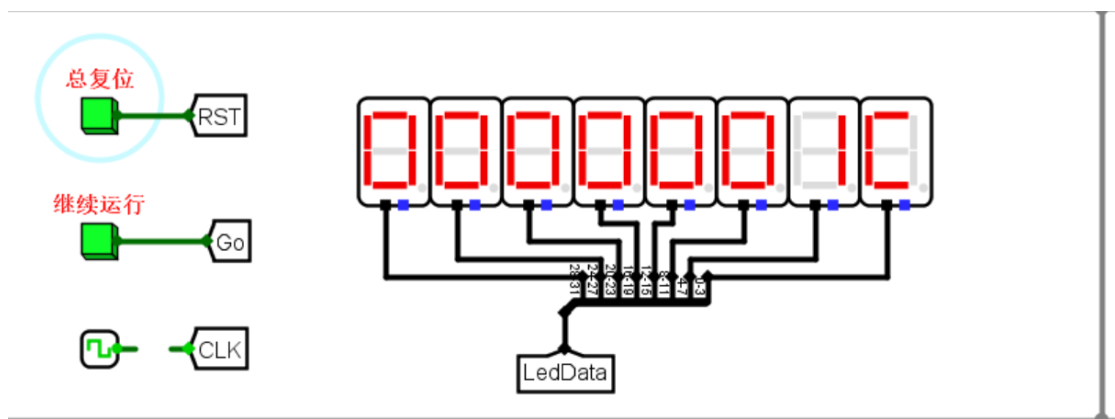
下列回答问题中包含。

- (2) 在气泡流水线 MIPS 处理器上运行测试程序。

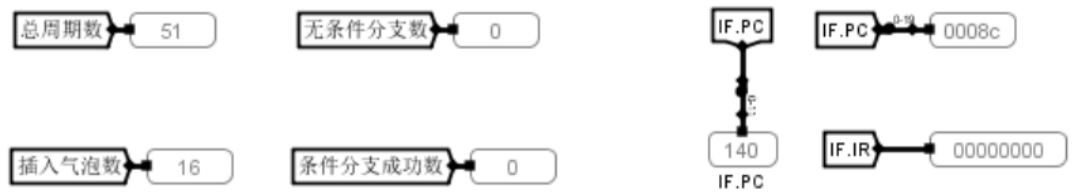
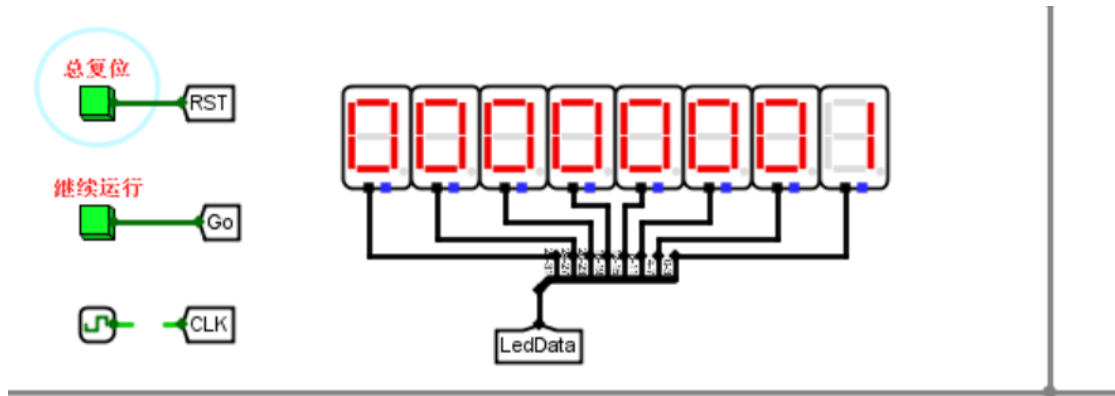
test2-1.hex: 数码管显示 1F，之后减 1、显示 1E，一直到 0，程序结束。



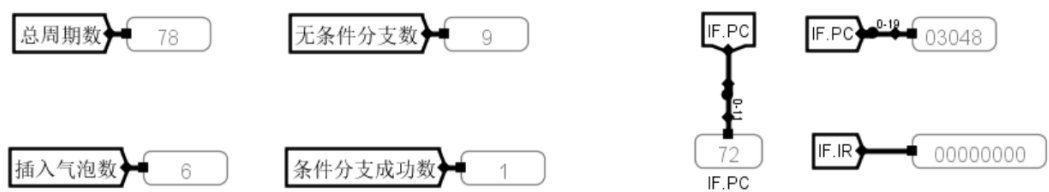
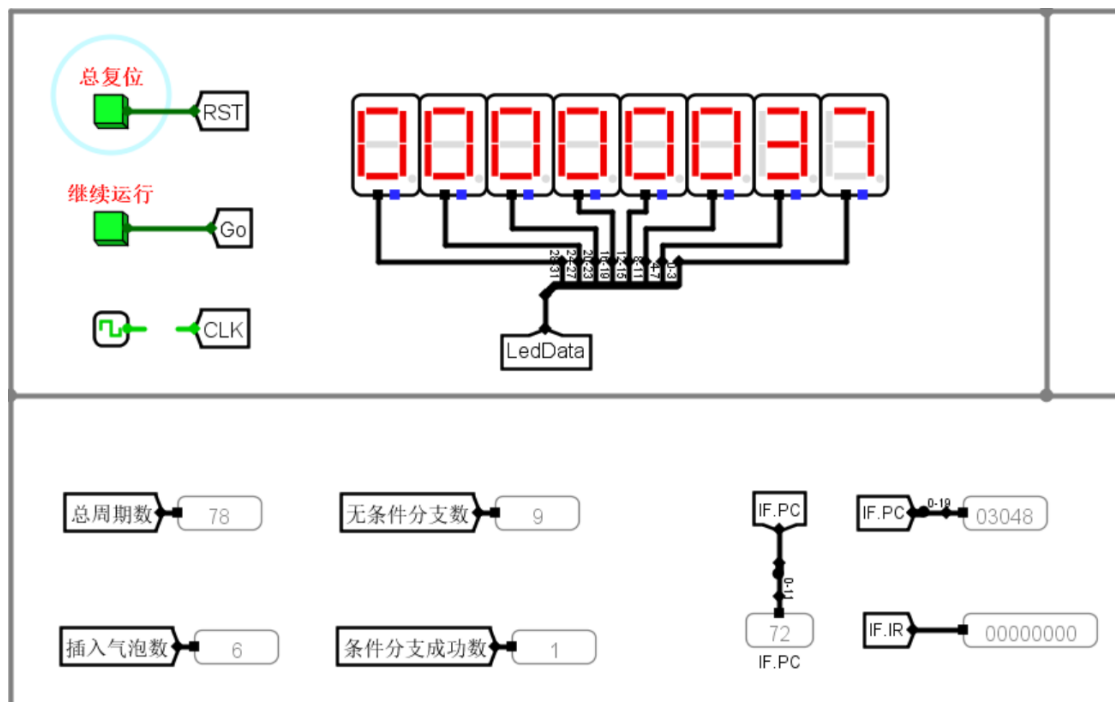
test2-2.hex: 数码管显示 1C。



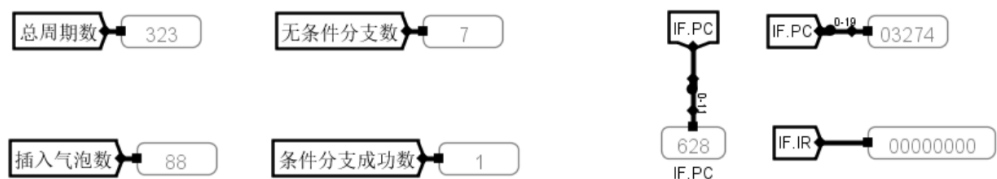
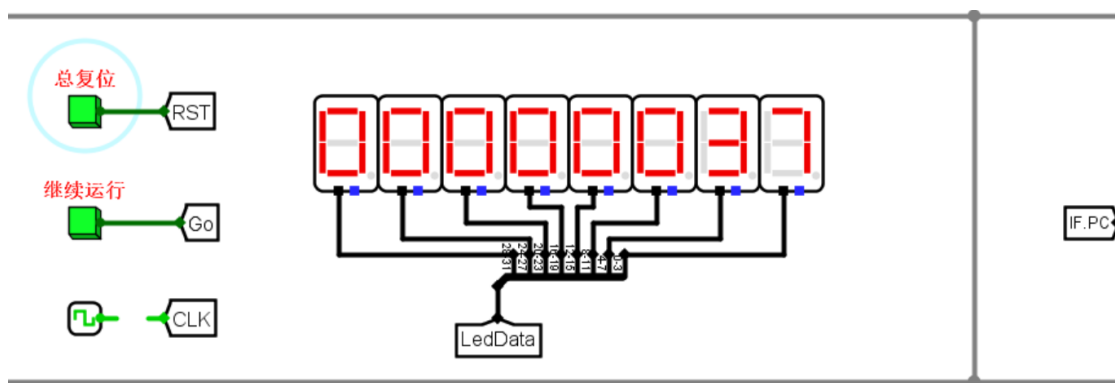
test2-3.hex: 数码管依次显示: 1、1、2、5、1。



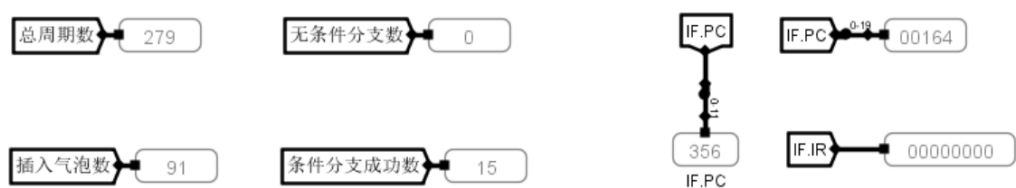
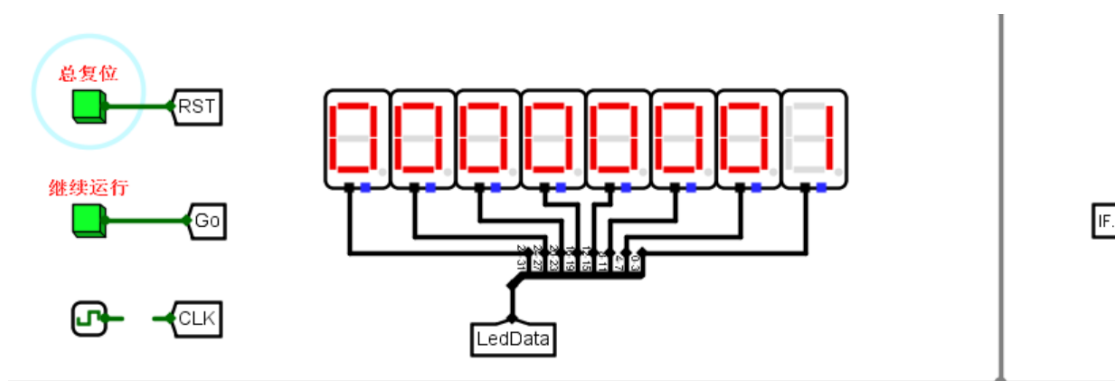
sum_mips.hex: 数码管显示 37H (55)。



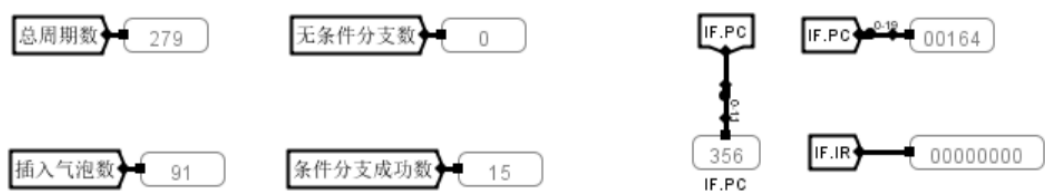
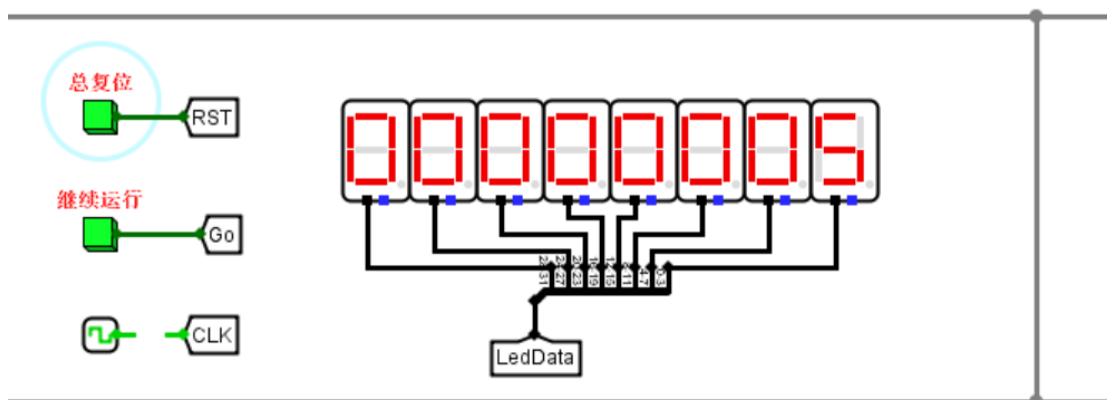
fib_mips.hex: 数码管依次显示: 0、1、1、2、3、5、8、d、15、22、37。



sort1_mips.hex: 数码管依次显示 5、4、3、2、1。



sort2_mips.hex: 数码管依次显示 1、2、3、4、5。



(2. 在气泡流水线 MIPS 处理器的数据通路上运行教材 P269 例 7.1 的程序 (test2-4.hex), 给出该程序运行后的时空图, 并与教材上的图 7.20 进行比较, 观测是否一致?

CLKs	取指令 IF	译码 ID	执行 EX	访存 EX	写回 WB
1	lw \$5,4(\$1)				
2	add \$6,\$5, \$7	lw \$5,4(\$1)			
3	sub \$1,\$2, \$3	add \$6,\$5, \$7	lw \$5,4(\$1)		
4	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble	lw \$5,4(\$1)	
5	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble	Bubble	lw \$5,4(\$1)
6	or \$7, \$6,\$7	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble	Bubble
7	and \$9,\$7, \$6	or \$7, \$6,\$7	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble
8	and \$9,\$7, \$6	or \$7, \$6,\$7	Bubble	sub \$1,\$2, \$3	add \$6,\$5, \$7
9		and \$9,\$7, \$6	or \$7,\$6, \$7	Bubble	sub \$1,\$2, \$3
10		and \$9,\$7, \$6	Bubble	or \$7, \$6,\$7	Bubble
11		and \$9,\$7, \$6	Bubble	Bubble	or \$7,\$6, \$7

观测一致。

(3.分析程序

test2-1.asm

功能：循环递减 s1 并显示，测试分支指令（j、beq、bne、jal、jr）。

关键：

对无条件跳转（j）和恒成立条件跳转（beq \$zero,\$zero），流水线预测跳转并清空后续错误指令（插入气泡），避免控制相关导致的停顿。bne 在译码阶段判断后，若跳转则清空预取指令，从正确地址执行。

test2-2.asm

功能：累加 0-7 并显示结果，测试寄存器读写的数据依赖（如 addi 写后 add 读）。

关键：

当后指令需读取前指令刚写入的寄存器流水线插入 nop 或停顿，等待前指令完成写回，消除 RAW 相关。通过硬件延迟确保访存顺序正确。

test2-3.asm

功能：通过逻辑和算术运算测试数据依赖。

关键：

指令间寄存器依赖（and \$s1 后 sub \$s1）通过插入气泡延迟后指令，避免读取未更新值。无依赖的逻辑运算（or \$s4,\$s5,\$s1）直接执行，验证流水线对独立指令的处理能力。

(3) 在重定向流水线 MIPS 处理器上运行测试程序。

在回答问题中有显示

(4) 在动态分支预测流水线 MIPS 处理器上运行测试程序。

回答问题：

(1) 理想流水线测试程序 test1-1.asm 中， 下述指令：

```
addi $s0, $zero, 0
```

```
ori $s0,$s0, 0
```

```
sw $ s0, 0($s0)
```

这三条指令为何不存在相关性？

答：第一条指令：addi \$s0, \$zero, 0，将寄存器 s0 设为 0。

第二条指令：ori \$s0, \$s0, 0，对 s0 执行逻辑或操作 ($s0 \mid 0$)，结果仍为 0。这条指令并未真正改变 s0 的值 ($0 \text{ OR } 0 = 0$)

第三条指令：sw \$s0, 0(\$s0)

将 s0 的值 (0) 存入内存地址 0。由于 s0 的值在第一条指令后已确定为 0，且第二条指令未改变该值，因此 sw 实际读取的 s0 与第一条指令的结果一致。实际未产生新数据，因此不构成真正的依赖。

(2) 理想流水线测试程序 test1-1.asm 中，

```
addi $v0,$zero,10
```

```
addi $s1,$zero,0  
addi $s2,$zero,0  
addi $s3,$zero,0
```

三条 addi 指令的作用是什么？为什么？

答：

(1. 指令总体作用是分别将寄存器 \$s1、\$s2、\$s3 初始化为 0

`addi $s1,$zero,0` : \$zero 是 MIPS 中一个特殊寄存器，其值恒为 0。这条指令将 \$zero (值为 0) 与立即数 0 相加，结果 0 存放到寄存器 \$s1 中，作用是将寄存器 \$s1 初始化为 0。

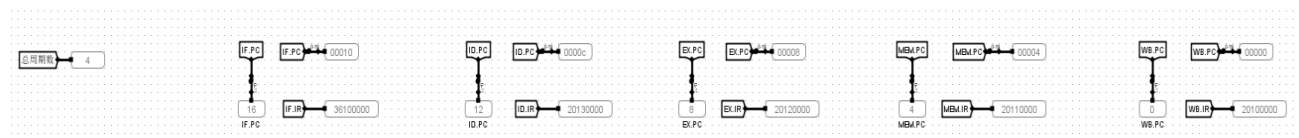
`addi $s2,$zero,0` : 同理，将 \$zero (值为 0) 与立即数 0 相加，把结果 0 存放到寄存器 \$s2 中，即初始化寄存器 \$s2 为 0。

`addi $s3,$zero,0` : 将 \$zero (值为 0) 与立即数 0 相加，结果存入寄存器 \$s3，实现将寄存器 \$s3 初始化为 0。

(2. 利用 `addi` 指令的运算功能，借助 \$zero 寄存器恒为 0 的特性，加上 0 这个立即数，从而把目标寄存器赋值为 0。

(3) 在理想流水线 MIPS 处理器的数据通路运行 `test1-1.hex`

- ①在数据通路的“指令存储器”中装入“test-1.hex”程序。按Ctrl+R，此时总周期数=0。
- ②接下去，按Ctrl+T二次，此时总周期数=1；再按Ctrl+T二次，此时总周期数=2；再按Ctrl+T二次，此时总周期数=3；再按Ctrl+T二次，此时总周期数=4。观看这期间IF.PC、IF.IR、ID.PC、ID.IR、EX.PC、EX.IR、MEM.PC、MEM.IR、WB.PC、WB.IR的变化情况，是不是与下一页的情况相同？



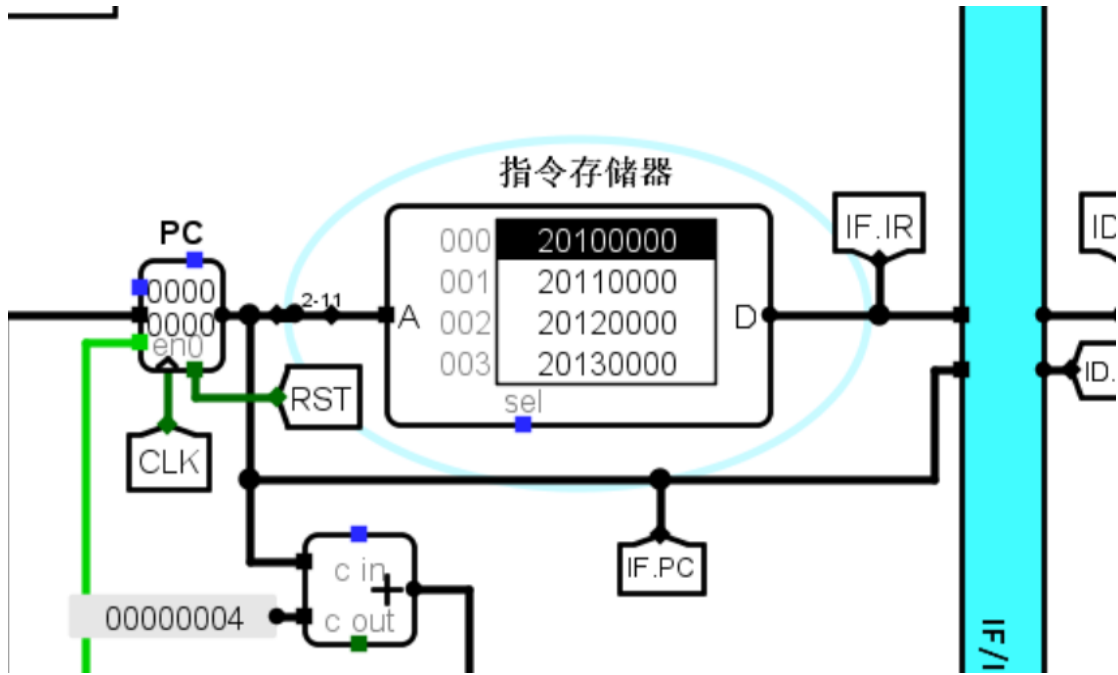
- ③接下去，按Ctrl+K，直到程序执行结束。观看总周期数是不是为21？观看“数据存储器”0-3单元的内容是不是为0-3？



上文中下一页即为实验七课件第 17 页的显示结果，需结合自己实际验证结果，观察总周期数每次递增的时候，各级流水线寄存器值的变化，从而理解流水线寄存器的作用，可截屏佐证并伴文字说明。

答：

(1.载入



(2.

T=1



T=2



T=3



T=4 一致



CTRL+K T=24

总周期数 21

一致

000 00000000 00000001 00000002 00000003 00000000 00000000

(4) 在气泡流水线中，对于测试程序 test2-1.asm，如何消除掉控制相关的？提示：

根据气泡流水线的下述控制信号并结合 test2-1.asm 中的某条分支指令

(j,beq,bne,jal,jr) 进行分析

IF/ID.CLR = BranchTaken

#出现分支跳转时，清空IF/ID段流水寄存器

ID/EX.CLR = Flush = BranchTaken + DataHazard

#出现分支跳转或数据相关时，清空ID/EX段流水寄存器

答：

在气泡流水线中，控制相关通过清空流水线 + 分支预测消除。以 bne \$zero,\$s1, jmp_next3 为例：

分支指令执行：bne 在译码阶段（ID）判断 s1≠0，需跳转至 jmp_next3。

控制信号触发：

(1. IF/ID.CLR=1 → 清空 IF/ID 寄存器（丢弃预取的错误指令，如 addi \$s1,\$zero,10)。

(2. ID/EX.CLR=1 → 清空 ID/EX 寄存器（防止错误指令进入执行阶段)。

插入气泡：清空寄存器后，流水线插入气泡（NOP），等待从 jmp_next3 地址重新取指。

通过强制清空错误指令 + 从正确地址取指，避免了分支指令对后续指令的影响，确保流水线无停顿运行。

(5) Test2-1.asm 中的下述 nop 空操作的作用是什么？

```

jmp_func:
    addi $s1,$s1,-1
    #   nop
    #   nop
    #   nop
    add $a0,$0,$s1
    #   nop
    #   nop
    #   nop
    syscall
    bne $s1,$zero,jmp_func
    jr $31
  
```

#a0=s1

#在数码管上从1F (32-1=31=1F) 开始显示，接下去减1，直到显示0

#子程序返回指令

答：

(1. 在 `addi $s1,$s1,-1` 修改 `s1` 后，需等待写回完成才能被 `add $a0,$0,$s1` 使用。`nop` 延迟后续指令执行，避免因寄存器未及时更新导致的数据错误。

(2. 分支指令（如 `bne $s1,$zero,jmp_func`）在译码阶段才确定跳转方向。`nop` 填充延迟槽，即使发生跳转，这些空操作也不会影响程序正确性，确保流水线无停顿运行。

(6) 针对 `test2-2.asm`，选择任意几条存在数据相关的指令，解释气泡流水线是如何消除这些相关性的？

答：以 `addi $s1,$s0,1` 和 `add $s0,$s0,$s1` 为例：

数据：`addi $s1,$s0,1` 指令写 `s1`，后条指令读 `s1`，存在 RAW 相关。

气泡流水线消除方法：插入气泡（NOP）：在两条指令间插入 `nop`，延迟后条指令执行，确保前条指令完成写回。

流水线停顿：通过控制信号暂停后续指令，等待 `s1` 更新后再继续执行，避免错误读取。

(7) 针对测试程序 `test2-3.asm` 中的下述 5

<code>and \$s1,\$s1,\$s2</code>	<code>#s1 = s1 ^ s2 = 1 ^ 1</code>	<code>=1</code>
<code>sub \$s2,\$s1,\$zero</code>	<code>#s2 = s1 - 0 = 1-0</code>	<code>=1</code>
<code>add \$s3,\$s1,\$s1</code>	<code>#s3 = s1 + s1 = 1+1</code>	<code>=2</code>
<code>or \$s4,\$s5,\$s1</code>	<code>#s4 = s5 s1 = 101 001 = 101</code>	<code>=5</code>
<code>and \$s5,\$s6,\$s1</code>	<code>#s5 = s6 ^ s1 = 101 ^ 001</code>	<code>=1</code>

条指令，解释如何利用气泡流水线解决数据相关性的？插入的气泡数与教材上的是否一致？

答：

(1. 当后续指令依赖前序指令的计算结果（如 `sub $s2,$s1,$zero` 需要 `and $s1,$s1,$s2` 的结果），而该结果尚未写入寄存器时，流水线会插入气泡（NOP 指令）暂停后续指令的执行，直到数据可用。气泡会占用流水线的时钟周期，确保指令按序正确执行。

(2.一致。

(8) 对于测试程序 `fib_mips.asm`，增加 `nop` 指令的目的是什么？尝试减少 `nop` 的数量，有何影响？

```

lw $a0,0($zero)
addi $v0,$zero,3
syscall
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop

```

答：

(1. 解决数据冲突：在 MIPS 流水线中，若一条指令需要前序指令的计算结果，但该结果尚未准备好，插入 nop 可暂停流水线，避免使用错误数据。

处理控制冲突：分支指令执行时，后续指令可能已进入流水线。插入 nop 可等待分支结果确定，防止错误执行预取的指令。

适配外设延迟：syscall 执行特殊操作时，插入 nop 确保操作完成后再执行后续指令。

(2. 数据冲突风险：减少 nop 可能导致指令提前执行，使用未就绪的数据，引发计算错误。

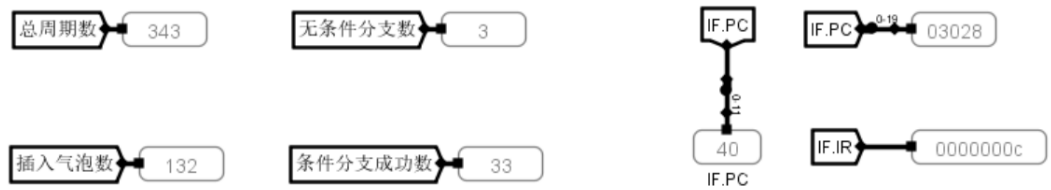
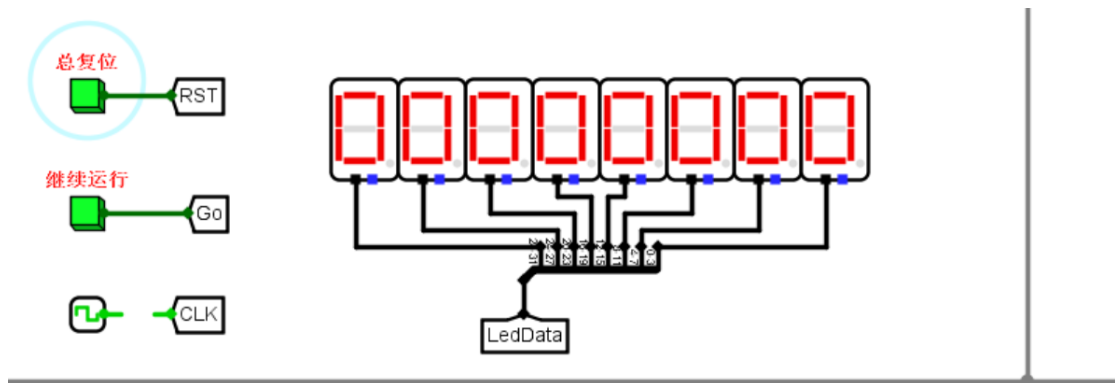
控制冲突风险：分支指令后 nop 不足，可能使流水线错误执行预取的指令，破坏程序逻辑。

性能可能下降：表面减少时钟周期，但冲突导致的重新执行增加整体耗时。

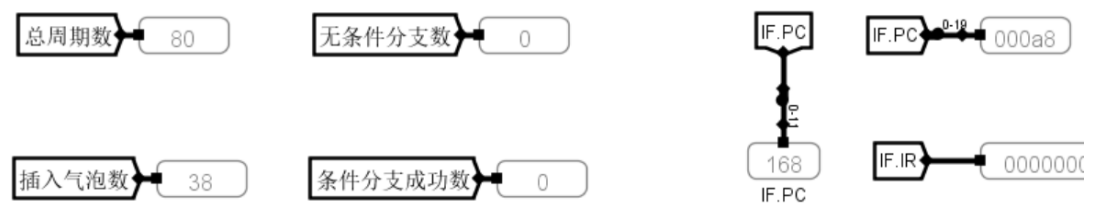
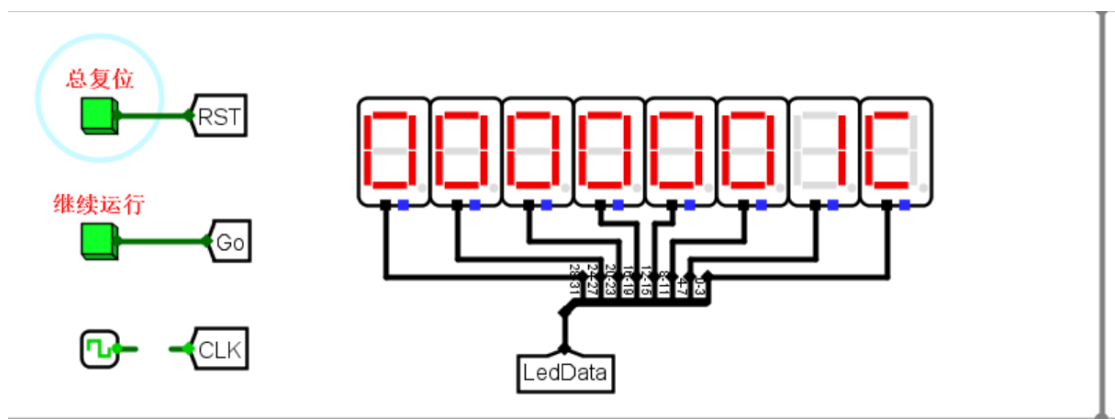
(9) 实验 PPT 第 28 页 要求 (1) (2)

答：

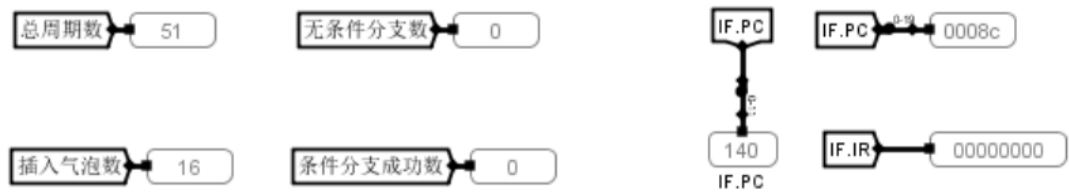
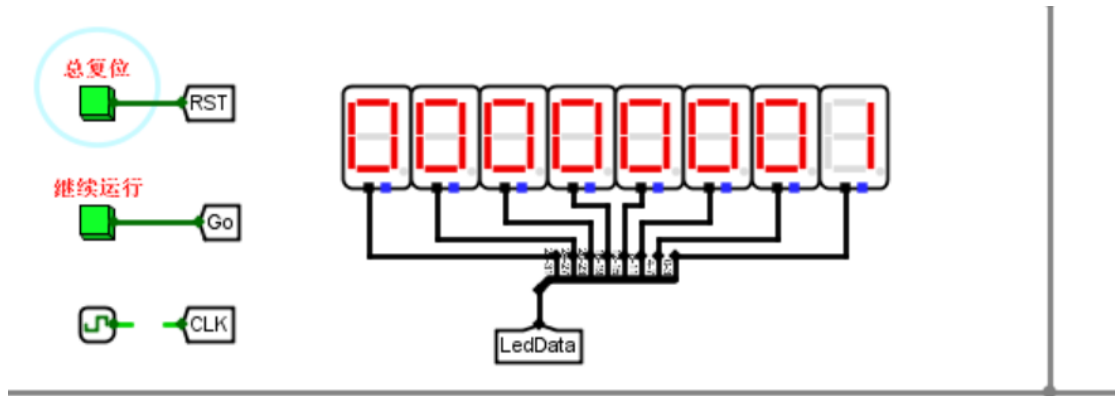
test2-1.hex：数码管显示 1F，之后减 1、显示 1E，一直到 0，程序结束。



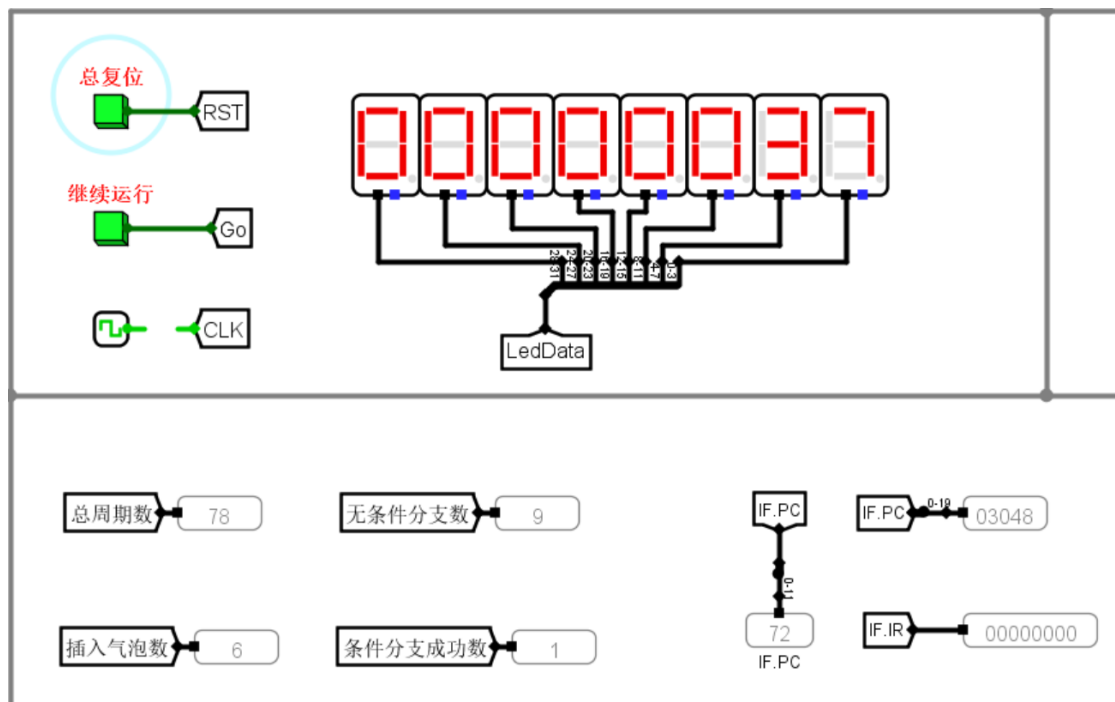
test2-2.hex: 数码管显示 1C。



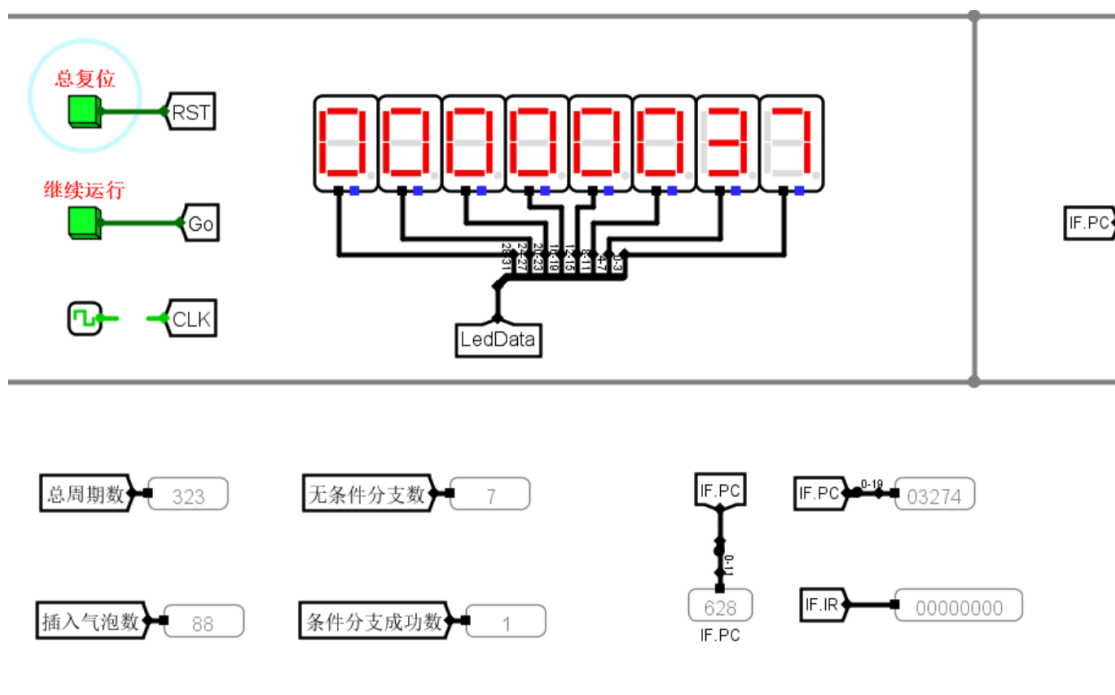
test2-3.hex: 数码管依次显示: 1、1、2、5、1。



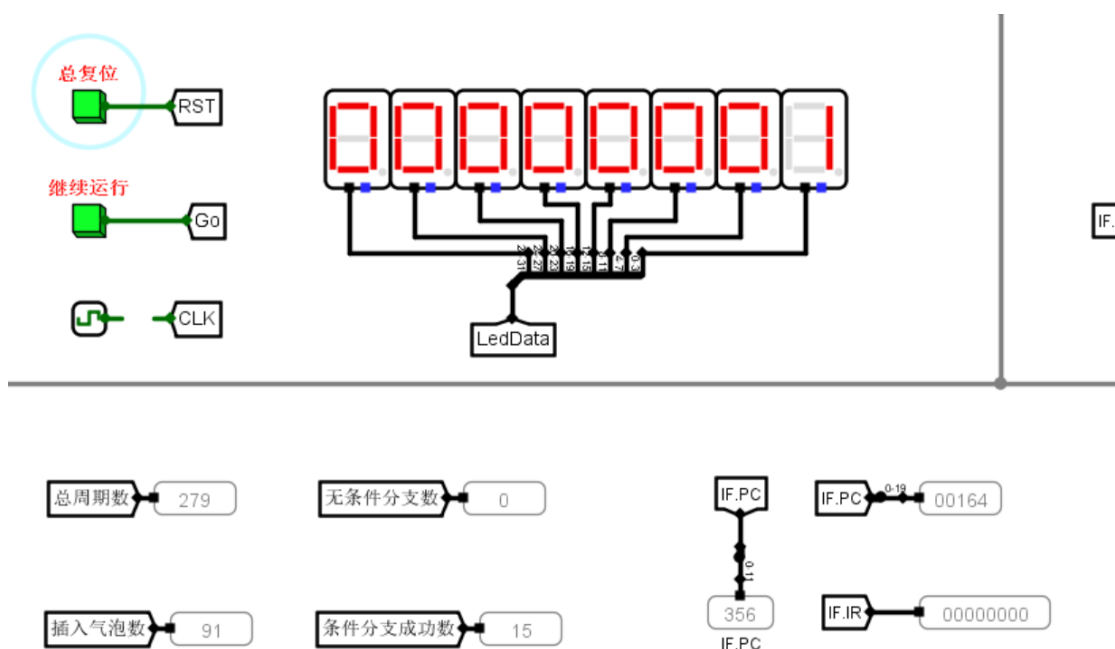
sum_mips.hex: 数码管显示 37H (55)。



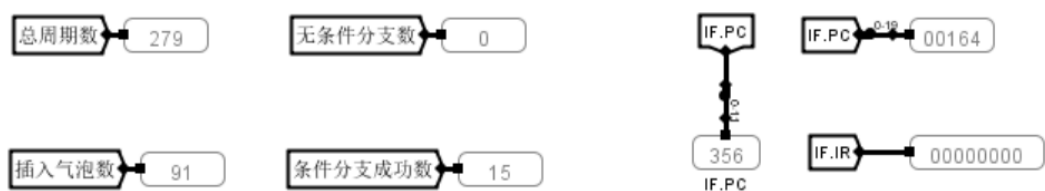
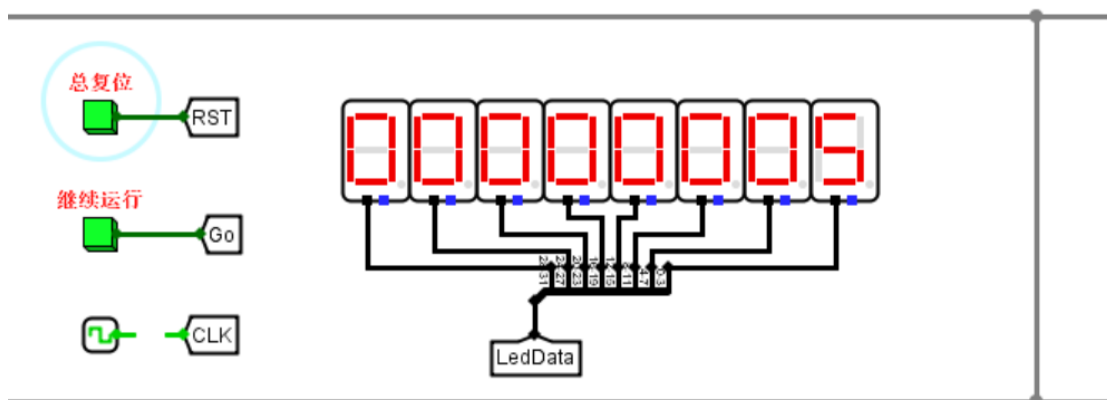
fib_mips.hex: 数码管依次显示: 0、1、1、2、3、5、8、d、15、22、37。



sort1_mips.hex: 数码管依次显示 5、4、3、2、1。



sort2_mips.hex: 数码管依次显示 1、2、3、4、5。



(2. 在气泡流水线 MIPS 处理器的数据通路上运行教材 P269 例 7.1 的程序 (test2-4.hex), 给出该程序运行后的时空图, 并与教材上的图 7.20 进行比较, 观测是否一致?

CLKs	取指令 IF	译码 ID	执行 EX	访存 EX	写回 WB
1	lw \$5,4(\$1)				
2	add \$6,\$5, \$7	lw \$5,4(\$1)			
3	sub \$1,\$2, \$3	add \$6,\$5, \$7	lw \$5,4(\$1)		
4	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble	lw \$5,4(\$1)	
5	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble	Bubble	lw \$5,4(\$1)
6	or \$7, \$6,\$7	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble	Bubble
7	and \$9,\$7, \$6	or \$7, \$6,\$7	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble
8	and \$9,\$7, \$6	or \$7, \$6,\$7	Bubble	sub \$1,\$2, \$3	add \$6,\$5, \$7
9		and \$9,\$7, \$6	or \$7,\$6, \$7	Bubble	sub \$1,\$2, \$3
10		and \$9,\$7, \$6	Bubble	or \$7, \$6,\$7	Bubble
11		and \$9,\$7, \$6	Bubble	Bubble	or \$7,\$6, \$7

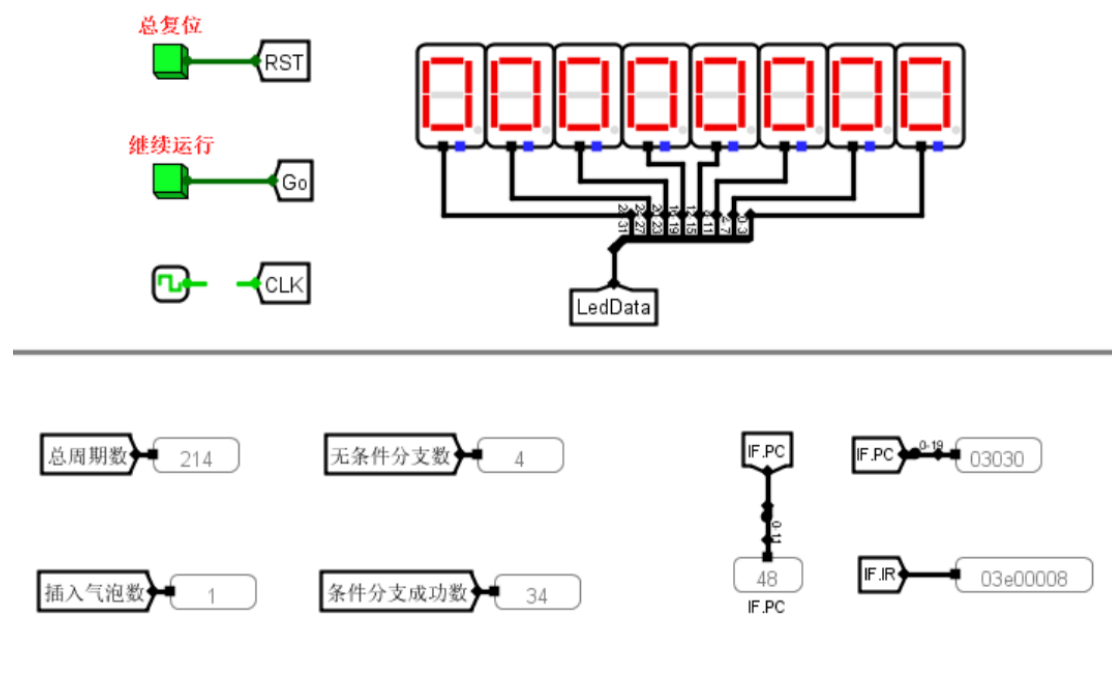
观测一致。

(10) 实验 PPT 第 42 页 要求 (1) (2)

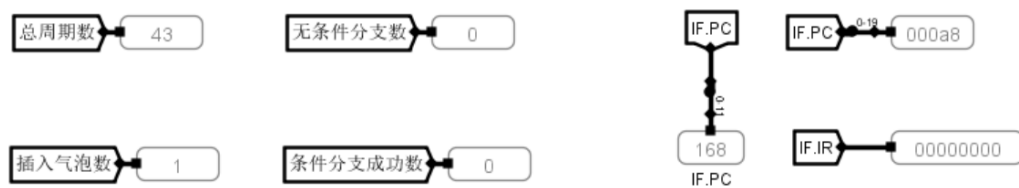
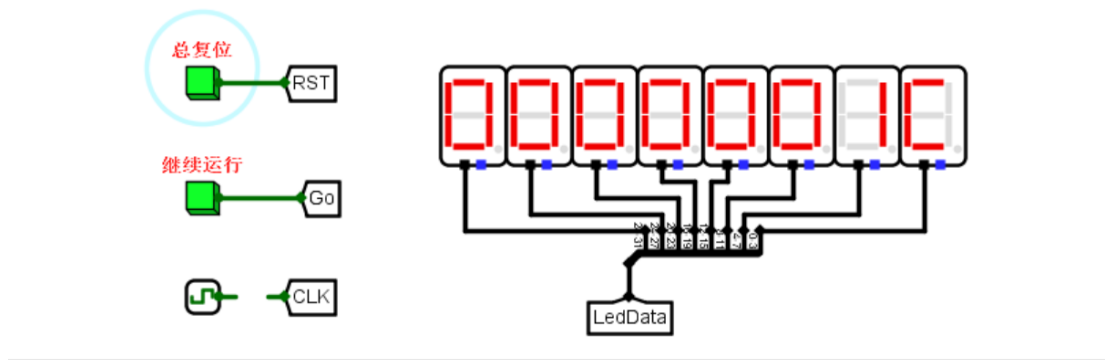
答：

(1.运行 test3-1.hex、test3-2.hex、test3-3.hex、sum_mips.hex、fib_mips.hex、sort1_mips.hex、sort2_mips.hex 等程序，记录每个程序运行后的总周期数、插入气泡数，并与气泡流水线对应程序的总周期数、插入气泡数进行比较，得出什么结论？

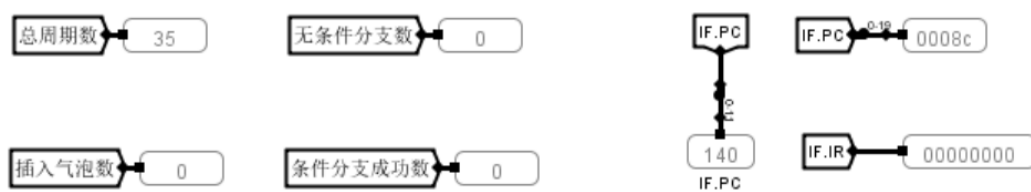
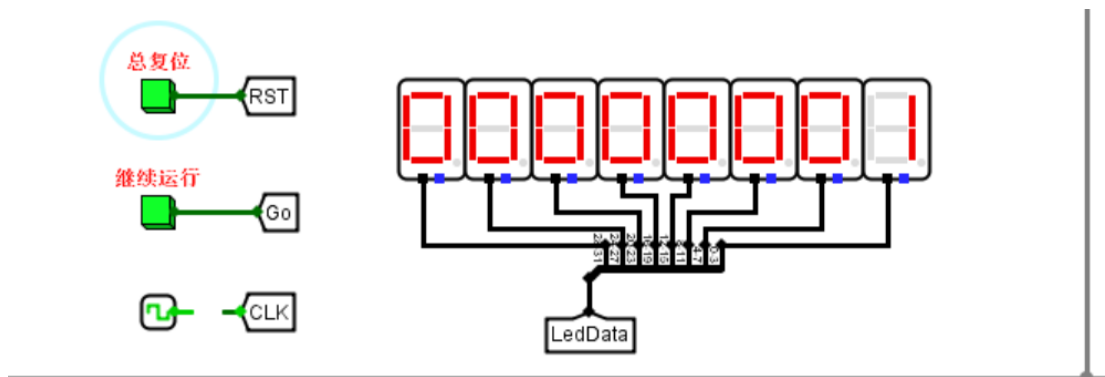
① test3-1.hex 程序执行后，过一段时间，数码管显示 1F，之后减 1、显示 1E，一直到 0，程序结束。



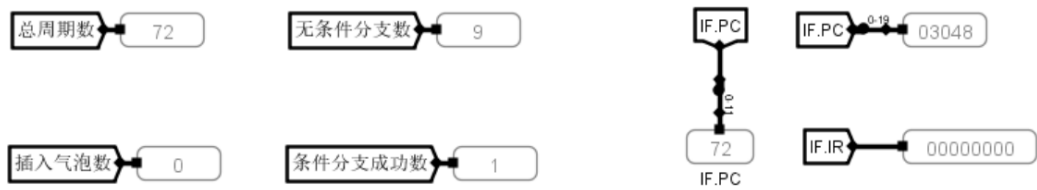
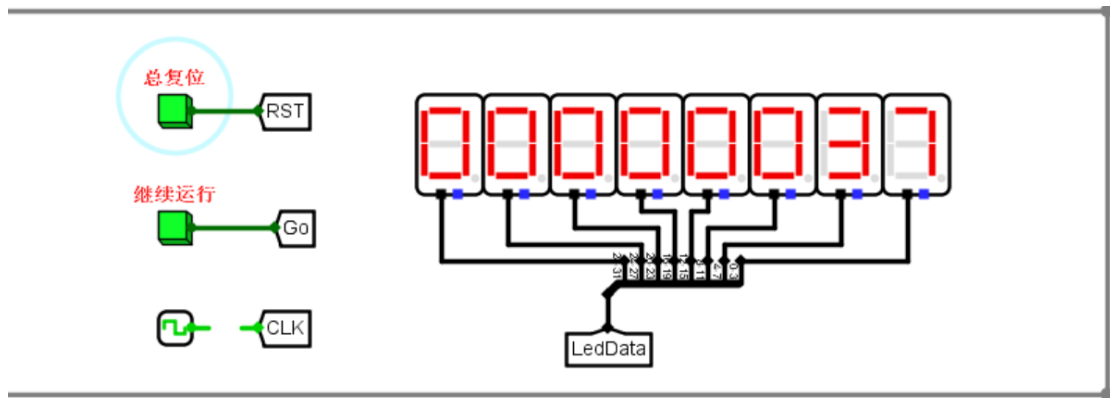
② test3-2.hex 程序执行后，过一段时间，数码管显示 1C。



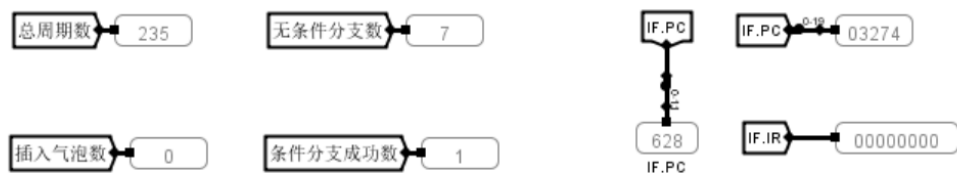
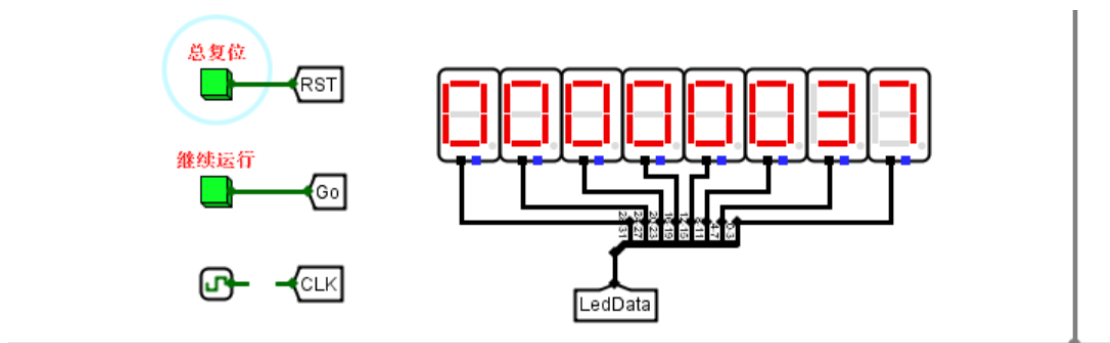
③ test3-3.hex 程序执行后，过一段时间，数码管依次显示：1、1、2、5、1。



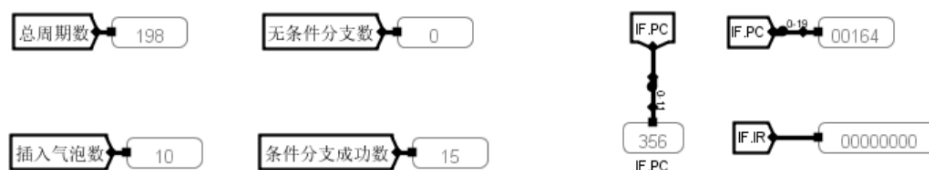
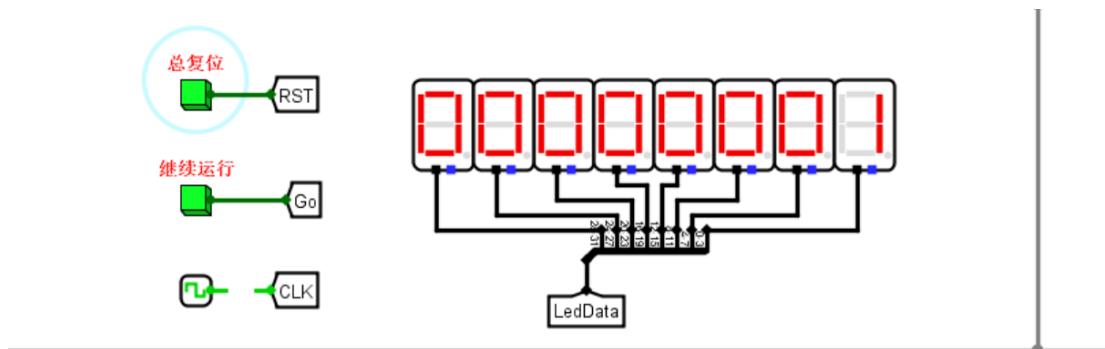
④ sum_mips.hex 程序执行后，过一段时间，数码管显示 37H (55)。



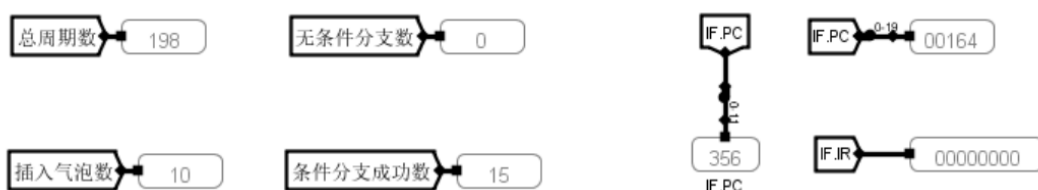
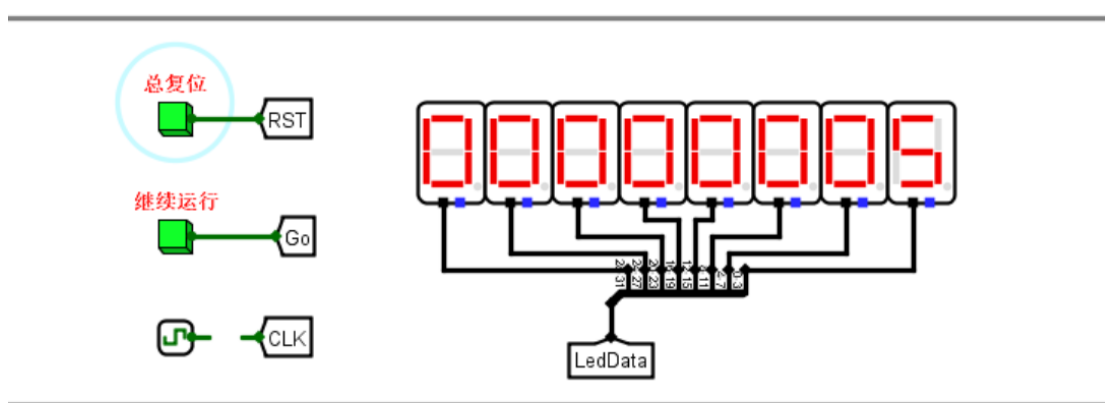
⑤ fib_mips.hex 程序执行后，过一段时间，数码管依次显示：0、1、1、2、3、5、8、d、15、22、37。



⑥ sort1_mips.hex 该程序执行后，过一段时间，数码管依次显示 5、4、3、2、1。



⑦ sort2_mips.hex 该程序执行后，过一段时间，数码管依次显示 1、2、3、4、5。



(2. 在重定向流水线 MIPS 处理器的数据通路上运行教材 P275 例 7.2 的程序(test3-4.hex)，给出该程序运行后的时空图，并与教材上的图 7.28 进行比较，观测是否一致？

- 首先装入 test3-4.hex 程序到“指令存储器”中。
- 按 Ctrl+R，此时总周期数=0，观看 IF.IR、ID.IR、EX.IR、MEM.IR、WB.IR 的值。
- 按 Ctrl+T 二次，此时总周期数=1，观看 IF.IR、ID.IR、EX.IR、MEM.IR、WB.IR 的值。
- 按 Ctrl+T 二次，此时总周期数=2，观看 IF.IR、ID.IR、EX.IR、MEM.IR、WB.IR 的值。

- 按 Ctrl+T 二次，此时总周期数=7，观看 IF.IR、ID.IR、EX.IR、MEM.IR、WB.IR 的值。
- 是不是与下二页的内容一样（即与教材上图 7.28 一致）？

答：

(1.

按 Ctrl+R，此时总周期数=0，观看 IF.IR、ID.IR、EX.IR、MEM.IR、WB.IR 的值



按 Ctrl+T 二次，此时总周期数=1，观看 IF.IR、ID.IR、EX.IR、MEM.IR、WB.IR 的值。



...



总周期数为 7



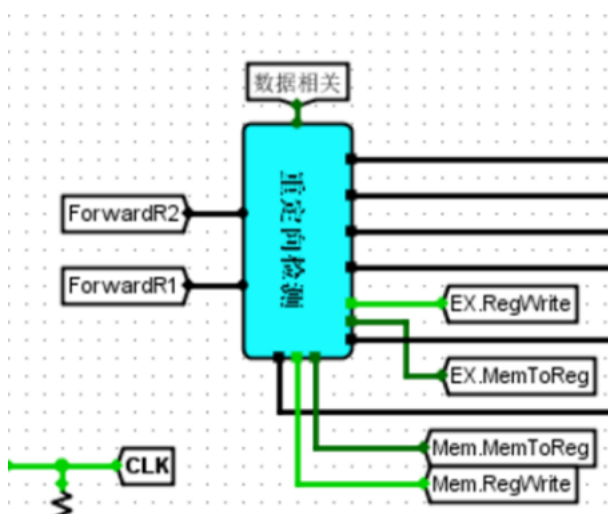
(2.与教材一致。

CLKs	取指令 IF	译码 ID	执行 EX	访存 MEM	写回 WB
1	lw \$5,4(\$1)				
2	add \$6,\$5, \$7	lw \$5,4(\$1)			
3	sub \$1,\$2, \$3	add \$6,\$5, \$7	lw \$5,4(\$1)		
4	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble	lw \$5,4(\$1)	
5	or \$7, \$6,\$7	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble	lw\$5,4(\$1)
6	and \$9,\$7, \$6	or \$7, \$6,\$7	sub \$1,\$2, \$3	add \$6,\$5, \$7	Bubble
7		and \$9,\$7, \$6	or \$7,\$6, \$7	sub \$1,\$2, \$3	add \$6,\$5, \$7
8			and \$9,\$7, \$6	or \$7, \$6,\$7	sub \$1,\$2, \$3

(11) 任选一个测试程序，基于气泡流水线和重定向流水线，结合其总周期数和插入气泡数以及电路，分析原因

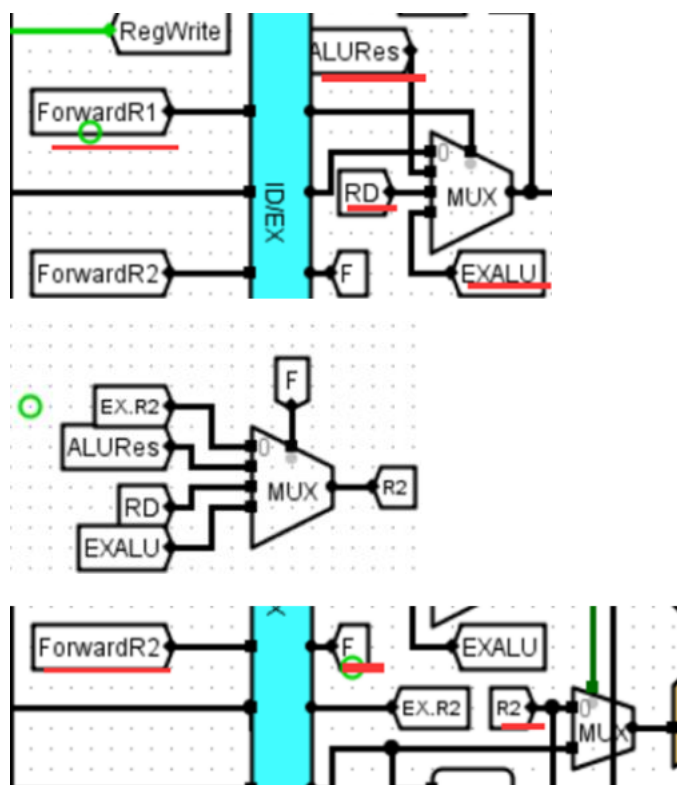
答：

重定向 MIPS 仅仅对与 LOAD 指令相关的数据冲突进行插入气泡，而对其他数据冲突采用重定向的技术以消除数据冲突的影响，除此之外重定向 MIPS 处理器的数据通路与气泡 MIPS 处理器的数据通路大致相同。



气泡 MIPS 处理器的数据通路中的重定向检测电路与重定向 MIPS 处理器的数据通路中的重定向检测电路实现的功能不完全相同。“数据相关”仅只会检测 LOAD 指令相关的

数据冲突；通过该电路可以得到 rs 和 rt 的重定向选择的条件（分别对应为 ForwardR1、ForwardR2，用于之后多路选择器的选择条件）。



(12) 动态分支预测流水线，结合讲解视频，深度理解分支预测原理以及 BHT 是如何预测的，又是如何更新的？

答：

动态分支预测利用分支指令过往执行的跳转情况（是否跳转），提前预判下次执行路径，让流水线提前取指，避免因等待分支结果而暂停，提升处理器效率。

BHT 预测逻辑

BHT 是存储分支指令历史执行结果的结构，以分支指令地址为索引，查找对应历史状态（如强跳转、弱跳转等），据此预测当前分支是“跳转”还是“不跳转”，指导流水线提前从预测路径取指。

BHT 更新逻辑

分支指令实际执行（如 EX/MEM 阶段）确定是否真跳转后，对比预测结果：

预测正确：微调 BHT 状态（如“弱跳转”强化为“强跳转”），巩固历史趋势。

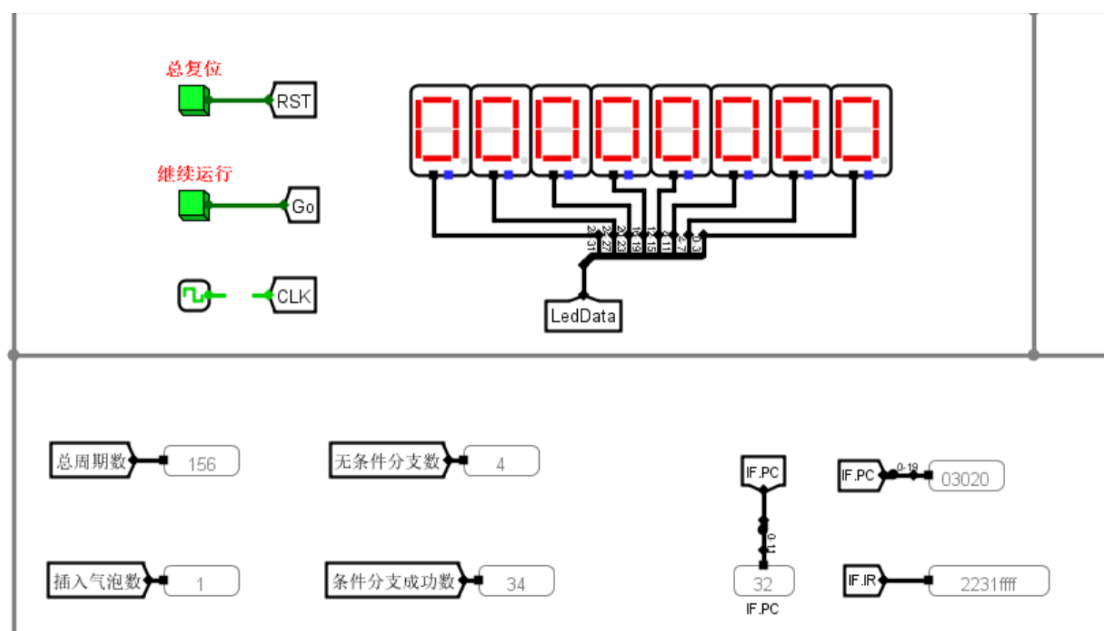
预测错误：直接修正 BHT 状态（如预测跳转但实际未跳转，就改为“弱不跳转”），保证后续预测更准。BHT 作为独立模块接入流水线，通过 PredictBr（触发预测）、BranchTaken（反馈实际结果）等信号，实现“预测-验证-更新”闭环。

(13) 实验课件 P57 要求 (1)

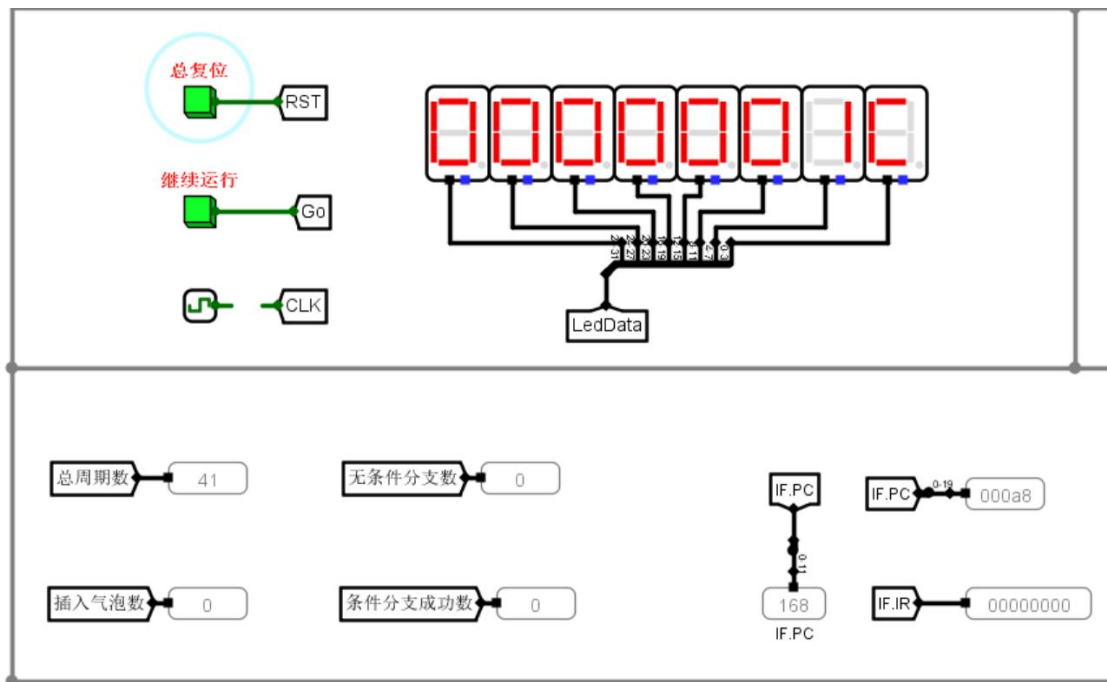
(1) 在动态分支预测流水线 MIPS 处理器的数据通路上运行 test4-1.hex、test4-2.hex、test4-3.hex、test4-

4.hex、sum_mips.hex、fib_mips.hex、sort1_mips.hex、sort2_mips.hex 等程序，记录每个程序运行后的总周期数、插入气泡数，并与气泡流水线、重定向流水线对应程序的总周期数、插入气泡数进行比较，得出什么结论？

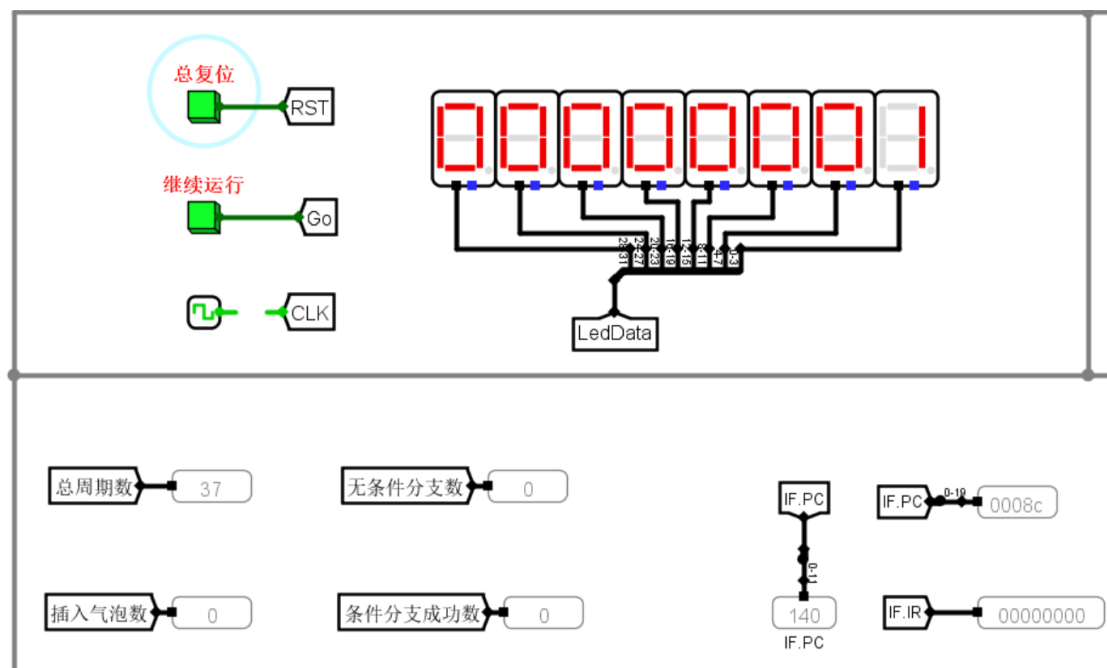
① test4-1.hex 程序执行后，过一段时间，数码管显示 1F，之后减 1、显示 1E，一直到 0，程序结束。



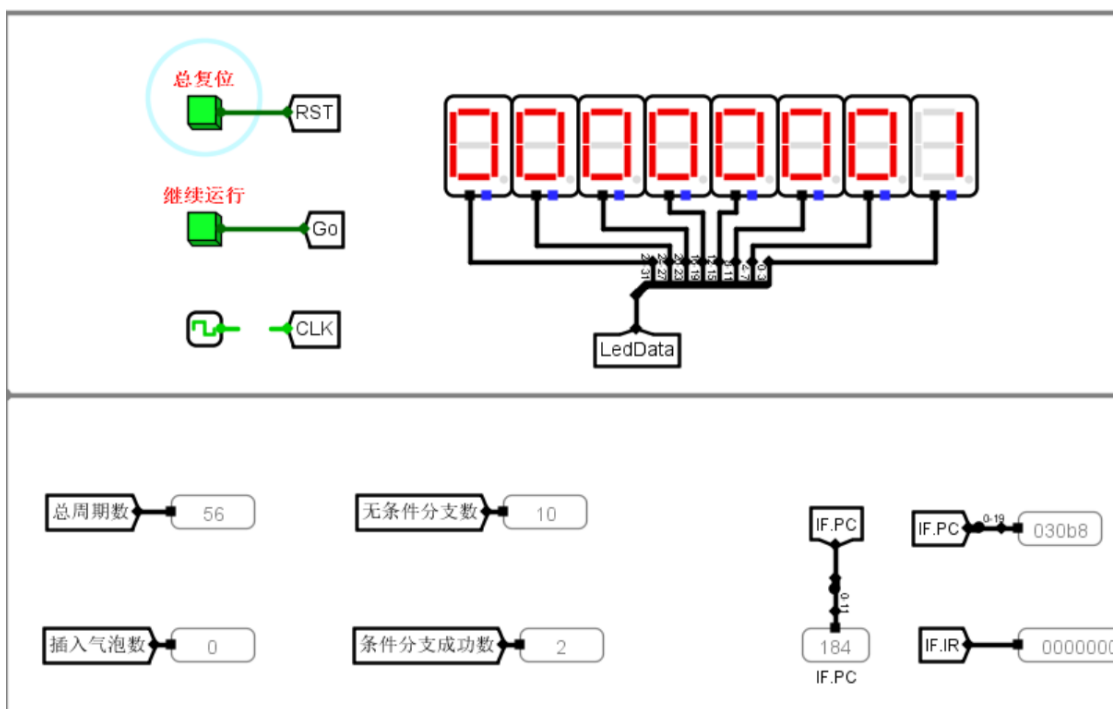
② test4-2.hex 程序执行后，过一段时间，数码管显示 1C。



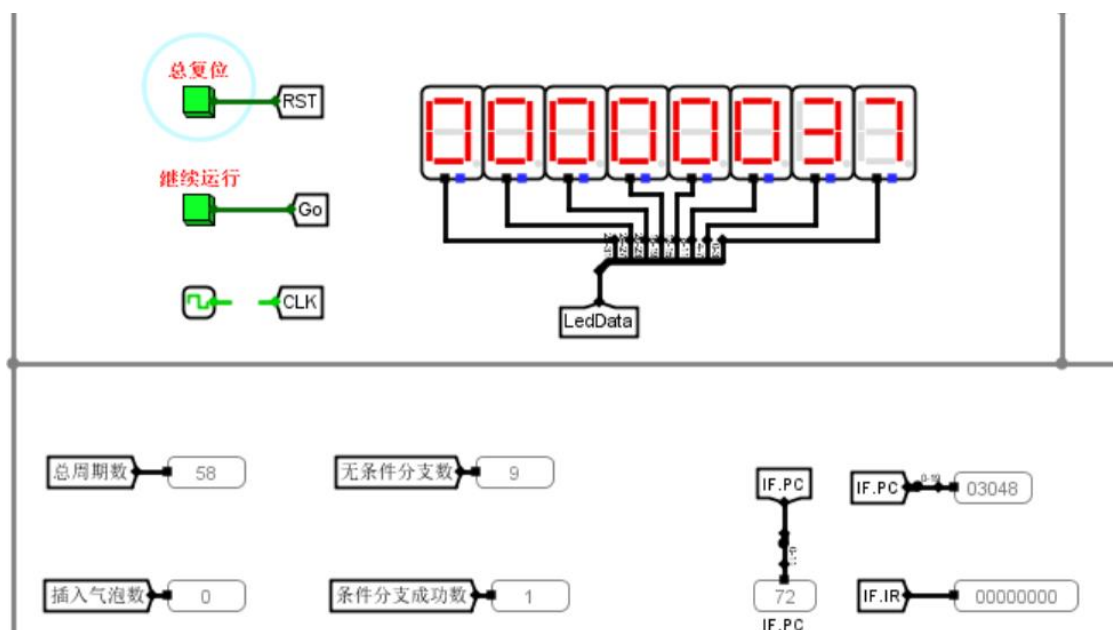
③ test4-3.hex 程序执行后，过一段时间，数码管依次显示：1、1、2、5、1。



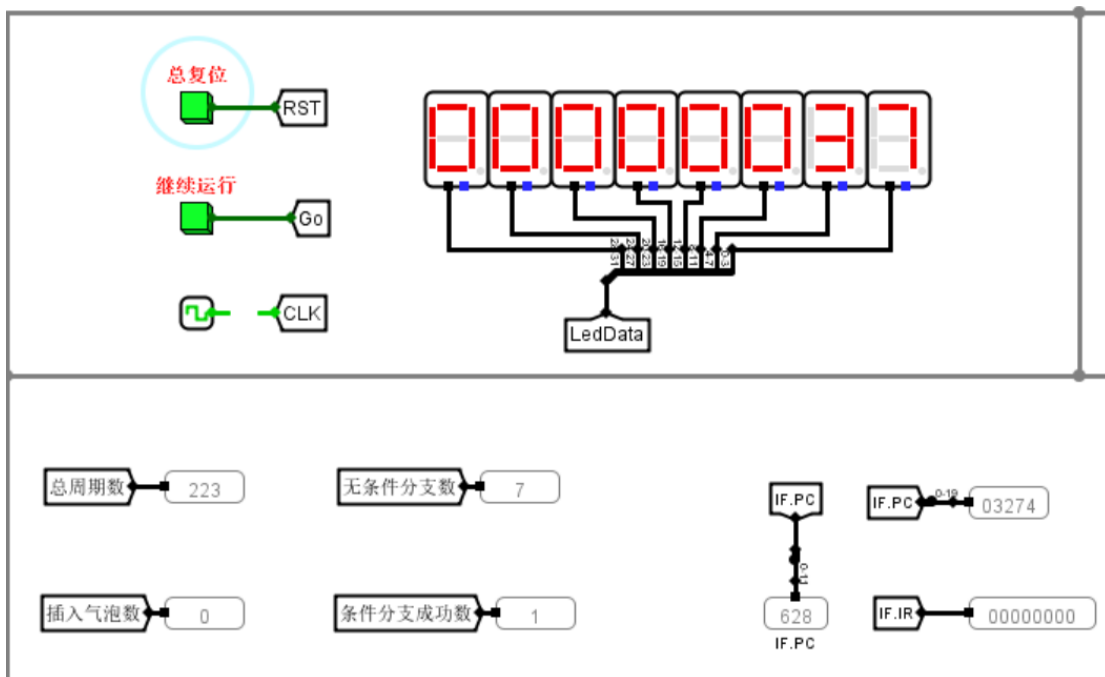
④ test4-4.hex 程序执行后，过一段时间，数码管显示：1。



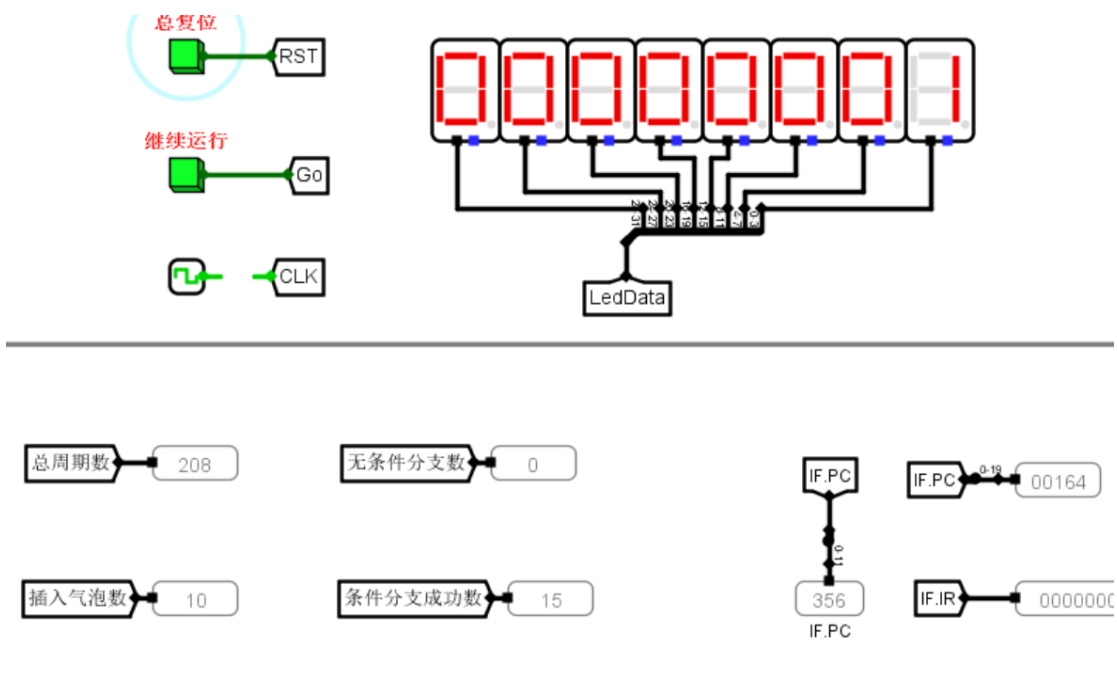
⑤ sum_mips.hex 程序执行后，过一段时间，数码管显示 37H (55)。



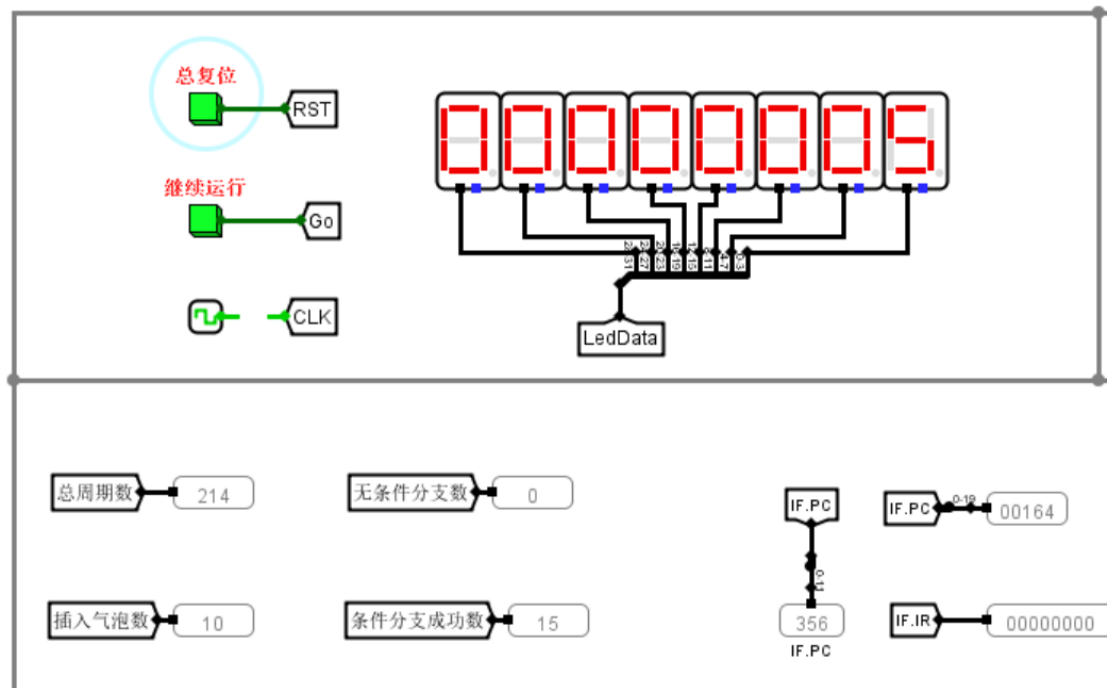
⑥ fib_mips.hex 程序执行后，过一段时间，数码管依次显示：0、1、1、2、3、5、8、d、15、22、37。



⑦ sort1_mips.hex 该程序执行后，过一段时间，数码管依次显示 5、4、3、2、1。



⑧ sort2_mips.hex 该程序执行后，过一段时间，数码管依次显示 1、2、3、4、5。



(14) 针对降序排序程序，观察其在气泡流水线，重定向流水线以及动态分支预测流水线的性能表现（总周期数和插入气泡数等）有何不同，并给出原因（可绘制三种情况下的流水线时空图进行解释）

答：

1. 气泡流水线

总周期数：最多

气泡数：大量

原因：

数据相关（如 lw 后立即使用数据）需插入气泡等待寄存器写入

分支指令（如 beq）需等待 EX 阶段计算结果，导致 2 个气泡

2. 重定向流水线

总周期数：中等

气泡数：较少

原因：

数据转发：通过硬件直接传递中间结果，减少等待

分支延迟槽：利用分支指令后的 1 条指令填充气泡

3. 动态分支预测流水线

总周期数：最少

气泡数：极少

原因：

分支预测：通过历史行为预测分支走向（如 bne），正确预测时无需停顿

预测错误代价：若预测错误需清空流水线（约 3-4 周期），但冒泡排序分支模式规律，准确率高

4. 实验报告提交

- (1) 实验报告命名为：学号+姓名+第 7 次实验报告.pdf。
- (2) 将实验报告上传到 course.xmu.edu.cn 上，第 7 次实验报告提交截止时间（2 周内）：**2025 年 6 月 4 日晚上 24 点。**